

SD TCL - Locks

The two kinds of lock, giving back the ones your session holds, and what you cannot do without an administrator.

There are two unrelated things called a lock, and knowing which is which is most of what this page has to say. They do not interact, and the names do not make that obvious.

database locks on a record or on a whole file. Taken by `readu`, `readl` and `filelock` in a program, and by `ed` while it holds a record

task locks 64 numbered flags, 0 to 63, with **no connection to any data**. A program takes one by agreement to keep other programs out of something

The statements a program uses to take either are in SD Basic - Locks and Transactions, and this page does not repeat them.

SD folds case, so a command may be typed in either case. Commands are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Only one lock verb is in an ordinary account

Inspecting locks and forcing them open are administrator verbs. `list.readu`, `list.locks`, `lock`, `clear.locks` and `unlock` are not in a standard or programmer account's VOC, so the names are not recognised at all. They are in the **administrator documentation**, under *Sessions and Locks*, which is a separate set your administrator may or may not have given you.

What every account has is `release`, which gives back locks this session holds. That is the whole of the TCL lock interface for an ordinary programmer, and the reason is defensible: a lock you took is yours to give back, and a lock somebody else took is not yours to look at or remove.

Giving your own locks back: `release`

```
release filename id {id ...}  
release filelock filename
```

It is the TCL form of the BASIC statement of the same name, needs no special rights, and acts **only on locks this session holds**. The file is named the way you would name it to any other verb.

```
:release
File not found
:release voc
Record id required
```

A record id is compulsory in the first form — there is no *release everything* at the prompt. `release filelock` *name* gives back a file lock, which has no id to give.

A `write` outside a transaction already releases the record lock, so `release` at the prompt is for the case where a program took a lock and ended without writing — usually because you stopped it — and for a file lock, which nothing releases implicitly.

What to do when something is stuck

You cannot see the lock table without the administrator verbs, so the useful sequence is:

- 1 `release` anything you know your own session took
- 2 `status` and `pstat`, on SD TCL - Processes and Phantoms, to see whether the program you think is holding it is still alive
- 3 if it is not yours, **report the file and record to an administrator** — they can list who holds it and force it open

A dead session's database locks are not released. A session killed from outside SD keeps both its user-table entry and its record and file locks, so everything wanting that record waits for a process that is not there. **Nothing an ordinary account can type will clear that**, and guessing at it wastes time — the recovery is an elevated `sd -cleanup`, which is an administrator's.

Task locks, and why you will rarely meet one

A task lock protects something that is **not a file** — a nightly job, an external resource, a sequence that must not run twice at once. Two programs agree that lock 7 means *this job*, and the number carries no other meaning.

Taking and releasing one is `lock` and `clear.locks`, both administrator verbs, so in practice task locks are taken from **inside a program** with the BASIC `lock` and `unlock` statements, which any account may compile and run. That is the route to reach for; the TCL verbs exist for an administrator inspecting or clearing the table by hand.

A task lock held by a killed session is not given back by `sd -cleanup`. It stays held, by a user number nothing is behind, until SD itself is restarted — a defect, and it is recorded in the project's fix lists. If a job guarded by a task lock will not start again after a crash, that is the first thing to suspect, and clearing it needs an administrator.

Who has these verbs

standard	<code>release</code>
administrator	<code>list.readu</code> <code>list.locks</code> <code>lock</code> <code>clear.locks</code> <code>unlock</code>

The split is between your locks and everybody's. Giving back what you hold is something any session may do. Looking at the machine's lock table, taking a numbered flag, or forcing another session's lock open are all administrator verbs, and `unlock` needs an elevated session on top of that.

See also

SD Basic - Locks and Transactions · SD TCL - Processes and Phantoms · SD TCL - Files and Records.